

MediClaim Orchestrator

The Workflows, Explained Simply

Every n8n workflow: what starts it, what it does, and why we need it

Multi-Agent Cashless and Reimbursement Claim Processing

With screenshots of the real workflows from the n8n editor

October 2026

Contents

1. What is n8n, and what is a workflow?.....3

 1.1 Why we use n8n.....3

 1.2 The golden rule: n8n manages, the API decides.....3

 1.3 How n8n and CrewAI work together.....3

2. Triggers: what starts a workflow.....4

3. The anatomy of a workflow.....6

 3.1 Each step in simple words.....8

 3.2 The node types you will see.....8

4. Hospital workflows (13).....9

5. Insurer workflows (19).....17

 5.1 The helper workflows (00-04).....17

 5.2 The six verification steps (11-16).....19

6. How the workflows are made and loaded.....26

7. Workflow settings (parameters).....28

8. Questions the examiner may ask about the workflows.....29

1. What is n8n, and what is a workflow?

n8n (say 'n-eight-n') is a tool for automating work. You build a **workflow** by joining boxes together. Each box is called a **node**, and each node does one small thing: receive a message, check something, call an API, wait, send a reply.

Think of a workflow as a **checklist that runs by itself**. When something happens, like a file being uploaded, n8n picks up the right checklist and goes through it step by step, without anyone pressing a button.

1.1 Why we use n8n

- **Claims take time.** A claim can wait hours for the insurer to reply, or days for a document. n8n is good at waiting, reminding and retrying.
- **You can see the process.** The workflow is a picture. Anyone can open it and understand what happens, even if they can't read code.
- **Things go wrong.** Networks fail, services restart. n8n retries, and one central error workflow catches every crash.
- **It is open source and free**, and it runs in Docker on our own laptop.

1.2 The golden rule: n8n manages, the API decides

The golden rule

n8n never makes a business decision. It never calculates money and never changes a claim's status by itself.

It only says **when** to do something. The **API** decides **what** happens, and the API checks that the claim is in the right state first.

So if a workflow runs twice by mistake, or stops halfway, the claim is never damaged. The API simply refuses the wrong step.

1.3 How n8n and CrewAI work together

CrewAI is the thinking inside one step: read this query, write this reply. **n8n** is everything around it: when to start, how long to wait for the agents, what to do if they're busy, who to tell if it fails. For example, when the insurer sends a query, n8n starts the hospital's agents, waits for their answer, and then hands it to the API.

2. Triggers: what starts a workflow

Every workflow starts with a **trigger**. We use four kinds.

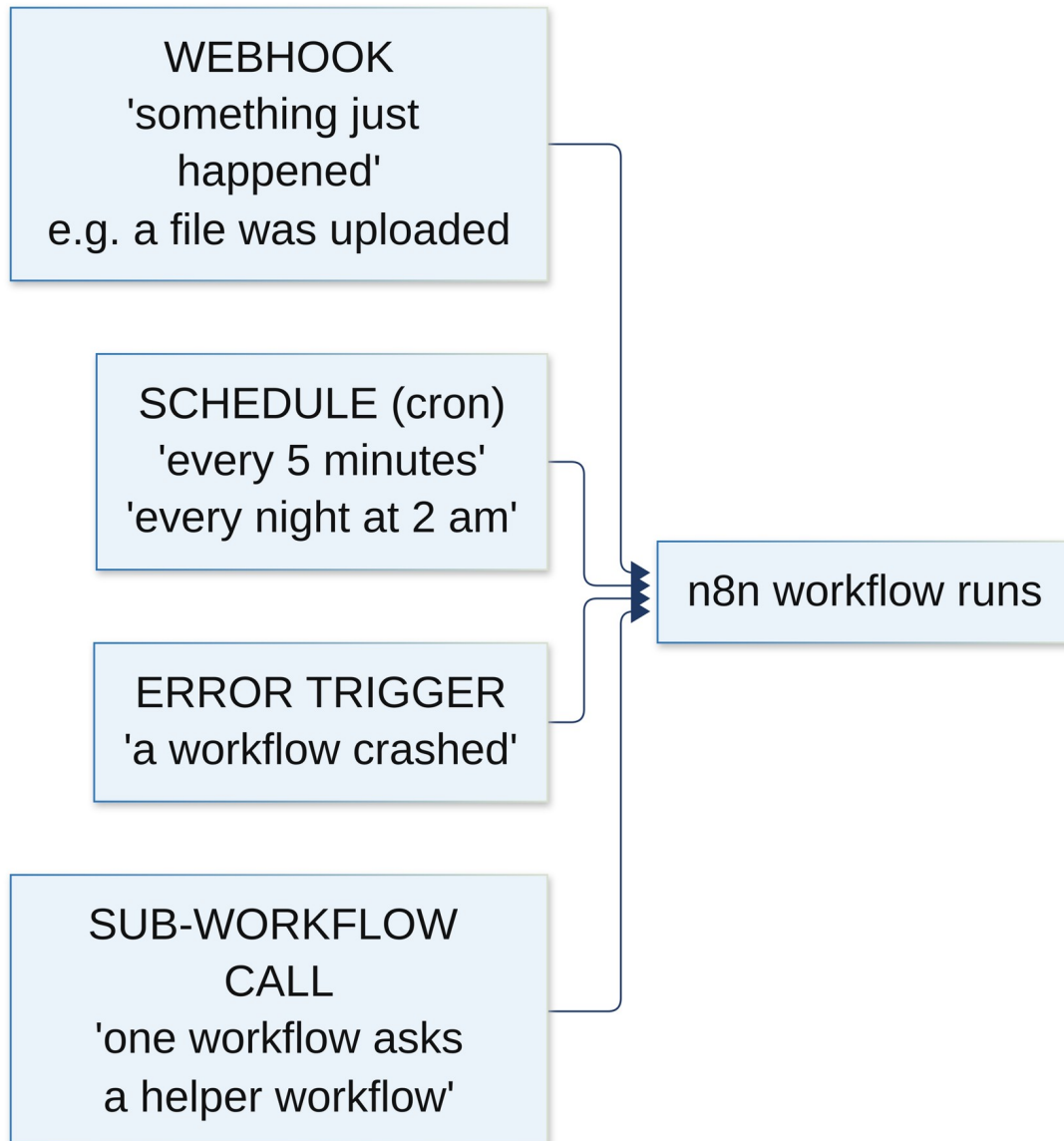


Figure 1: The four kinds of trigger

Trigger	Simple meaning	Where we use it
Webhook	A web address. When the API calls it, the workflow starts. It means 'something just happened'	Upload, claim submitted, claim arrived, query, decision, settlement
Schedule (cron)	A clock. It starts the workflow again and again	Every 2 min (sweeper), every 5 min (reminders, stuck builds, query timers), every hour (late replies), 2 am every night (audit check)
Error trigger	Starts when any other workflow crashes	One error workflow on each side
Sub-workflow	A workflow called by another	Get token, duplicate check, call

Trigger	Simple meaning	Where we use it
(Execute Workflow)	workflow, like a helper function	API, call agents, each verification step

The scheduled workflows also have a webhook (like **jobs/reminders**), so we can run them right away in a demo or a test instead of waiting for the clock.

3. The anatomy of a workflow

Almost every workflow we have starts the same way. Once you understand this pattern, you understand most of them.

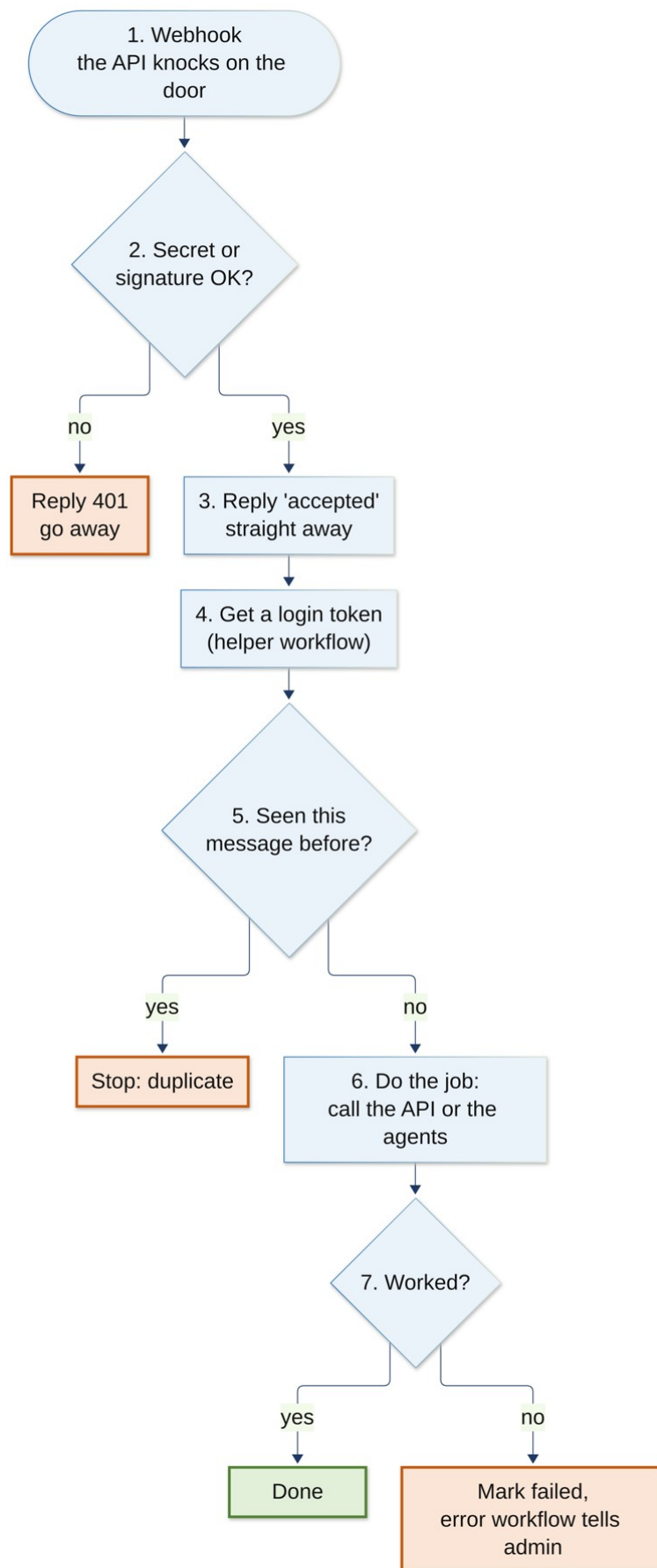


Figure 2: The standard pattern used by our workflows

3.1 Each step in simple words

1. **Webhook.** The API calls the workflow's web address and sends the case ID.
2. **Secret or signature check.** On the hospital side the call must carry the shared secret (**X-Webhook-Secret**). On the insurer side the message must have a valid HMAC signature. If not, n8n answers **401** and stops.
3. **Reply straight away.** n8n says 'accepted' (200 or 202) at once, so the API doesn't sit waiting. The real work continues in the background.
4. **Get a login token** from Keycloak, using the helper workflow. The token is kept for 4 minutes so we don't ask every time.
5. **Duplicate check.** Every message has an **idempotency key** (a unique ID). If we've seen it before, stop. So nothing happens twice.
6. **Do the job.** Call the API, or call the agents, and wait for the result.
7. **If it fails,** mark the job as failed. The error workflow also tells the admin.

3.2 The node types you will see

Node	What it does
Webhook	Receives a call from the API
Schedule Trigger	Starts on a timer
IF	Asks a yes/no question and takes one of two paths
Respond to Webhook	Sends the answer back to whoever called (200, 202 or 401)
HTTP Request	Calls one of our services (API, agents, document pipeline...)
Execute Workflow	Calls a helper workflow
Wait	Pauses: 3 seconds, 30 seconds, 15 minutes, 8 hours
Set / Code	Prepares the data for the next node
Split in Batches	Handles a list a few items at a time (for example, 10 reminders at a time)
Error Trigger	Starts when another workflow crashes
Sticky note	A yellow note in the editor that explains the workflow's purpose

In the screenshots, the small red triangles on some nodes only mean the login details are filled in when the container starts. That is expected.

4. Hospital workflows (13)

The hospital side has 10 workflows and 3 small helper workflows. They run in the hospital's own n8n, on port 5688.

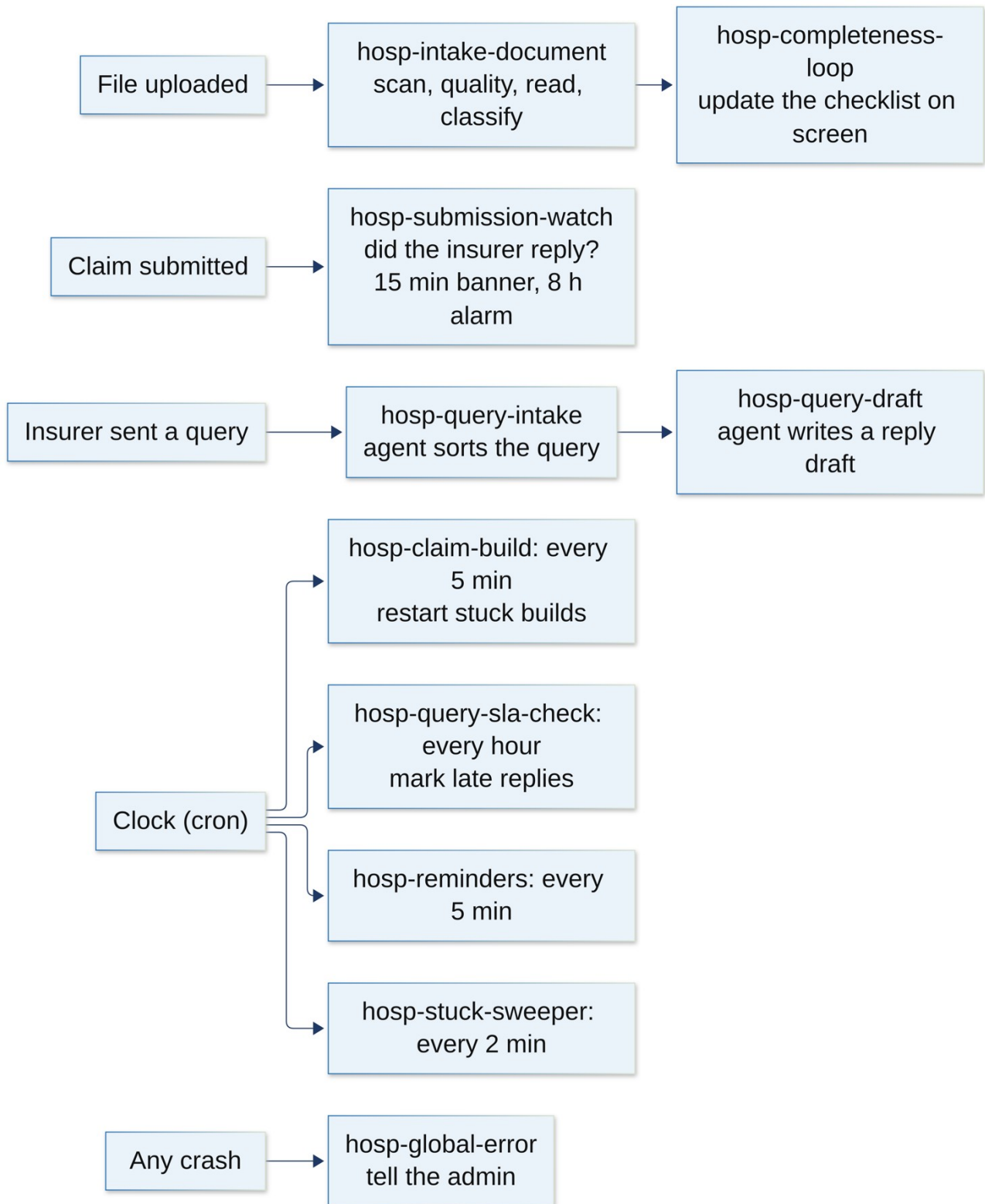


Figure 3: Hospital workflows: what starts each one

hosp-intake-document

Trigger: webhook `intake/document-uploaded`, every time the desk uploads a file.

What it does:

- Check the secret, reply 'accepted', get a token, drop duplicates.
- Get the document. If the virus scan is not clean, stop.
- Ask the vision service: is the photo good enough? If it's too blurry or dark, mark it **needs review**.
- Send it to the document pipeline to read the text (OCR) and find the fields. Wait for the result, checking a few times.
- Save the reading twice (two passes, to compare them), then classify the document: bill, discharge summary, ID card and so on.
- If anything fails, mark the document **failed** so the desk can see it.

Why we need it: This is the front door. Every document goes through the same safe steps before anyone or any agent uses it.

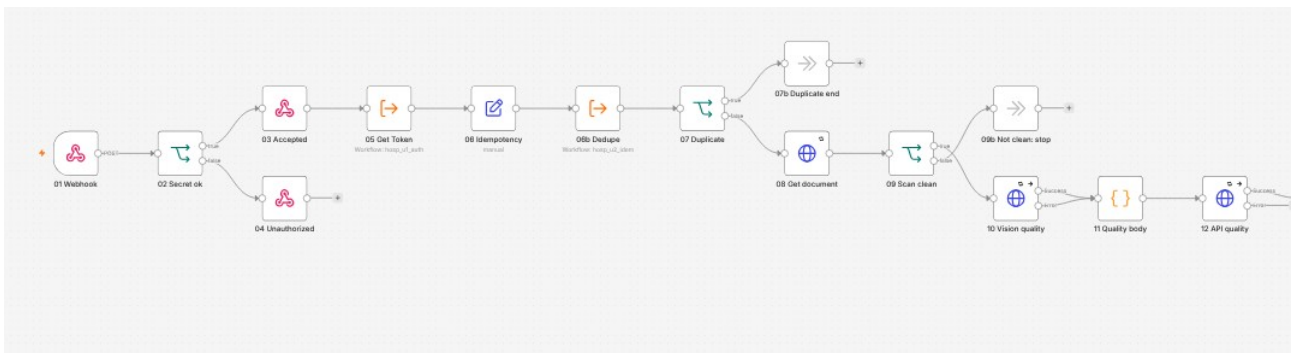


Figure 4: hosp-intake-document, first half: webhook, secret, duplicate and virus-scan checks

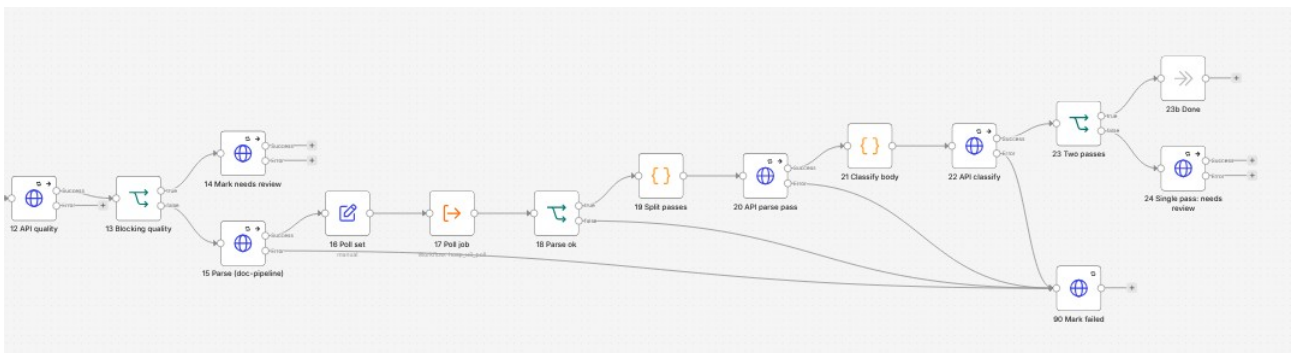


Figure 5: hosp-intake-document, second half: quality, parse with polling, classify

hosp-completeness-loop

Trigger: webhook **completeness/changed**, whenever the checklist may have changed.

What it does:

- The usual checks, then tell the screen to refresh the checklist.
- The API itself works out what is missing; n8n only passes on the news.

Why we need it: So the desk sees straight away which documents are still missing.

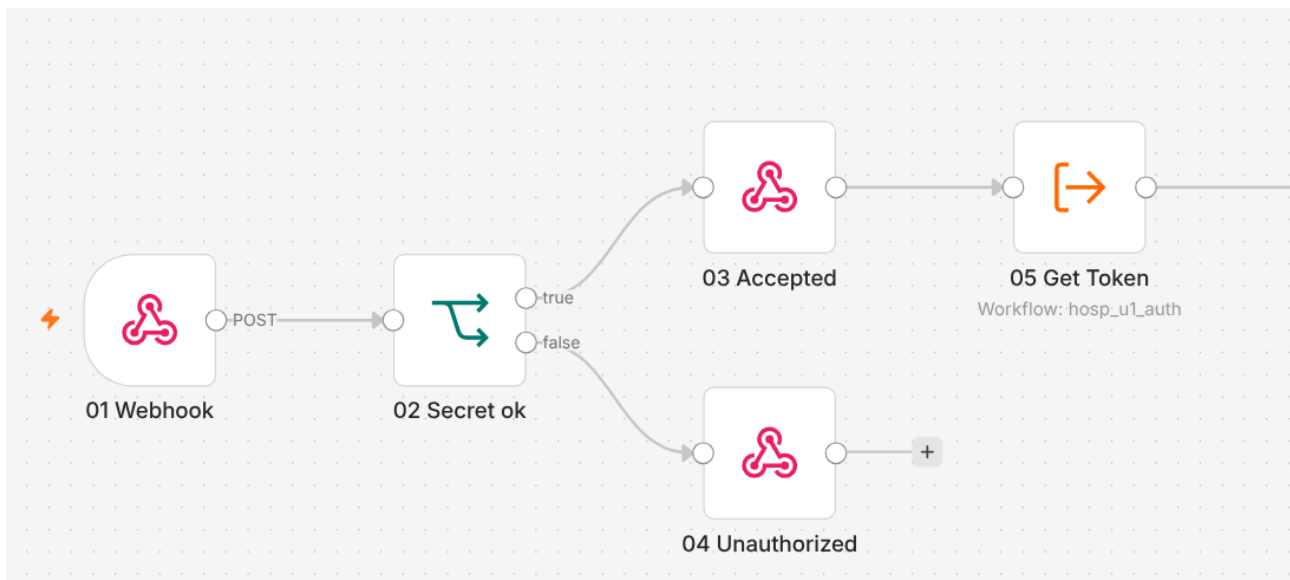


Figure 6: hosp-completeness-loop

hosp-claim-build

Trigger: a clock every 5 minutes (and webhook `jobs/claim-build`).

What it does:

- The API starts the CrewAI ClaimFlow itself, as soon as all documents are in.
- This workflow is the safety net: it finds builds that got stuck (for example, the agents were restarted) and puts them back so they run again.

Why we need it: So no claim is ever stuck in 'building' forever.

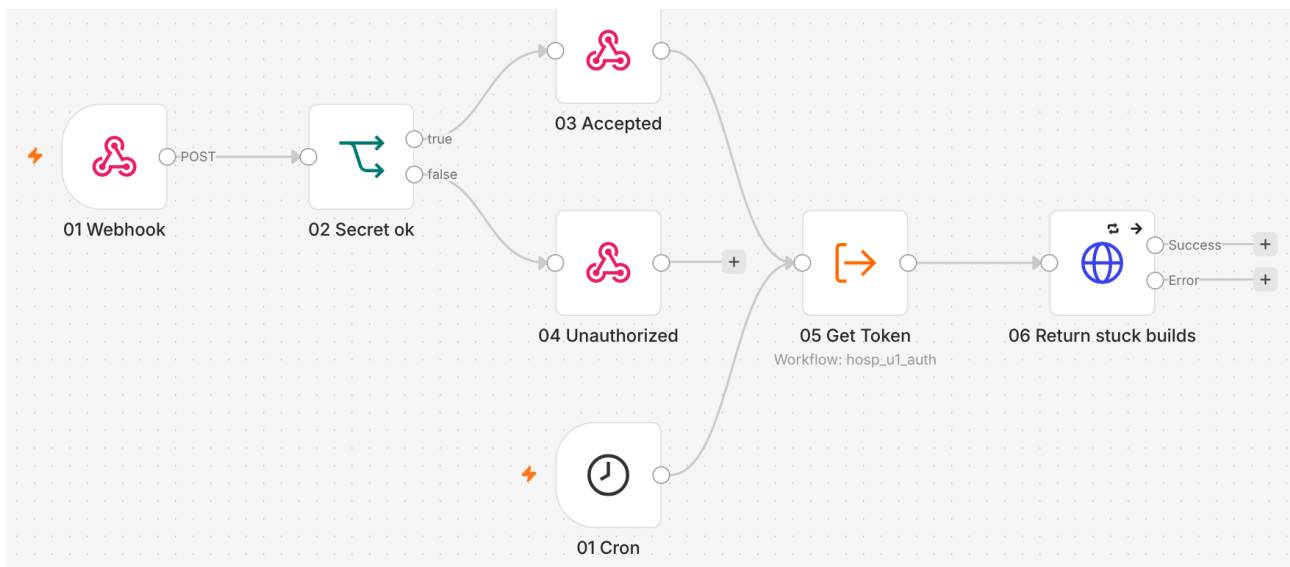


Figure 7: hosp-claim-build: clock path and manual webhook path

hosp-submission-watch

Trigger: webhook `claim/submitted`, when the officer submits a claim.

What it does:

- Wait 15 minutes, then ask: did the insurer acknowledge the claim?
- If not, show a banner on the screen.

- Wait until 8 hours, ask again. If there is still no reply, raise a high-priority alert for the operations team.

Why we need it: A claim sent into silence is a real risk: the patient may be stuck waiting. This makes sure somebody notices.

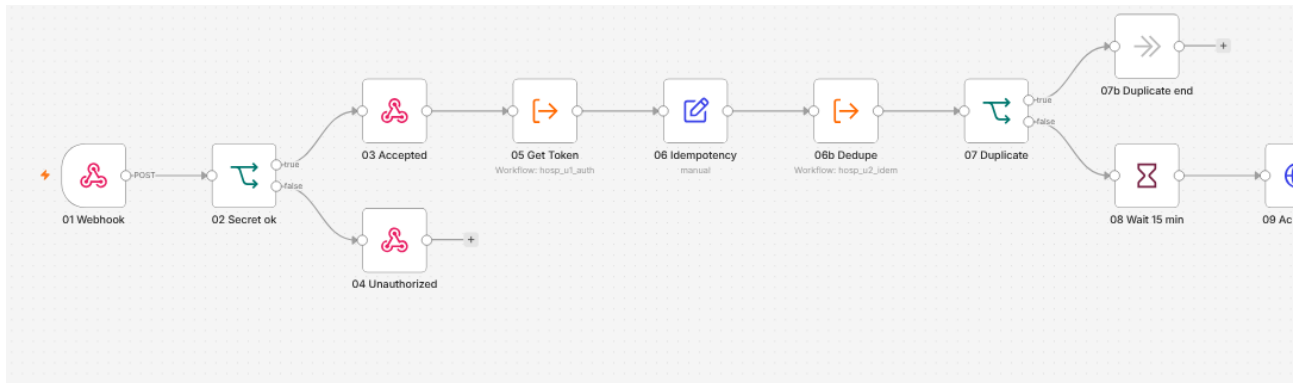


Figure 8: hosp-submission-watch, first half: wait 15 minutes and check

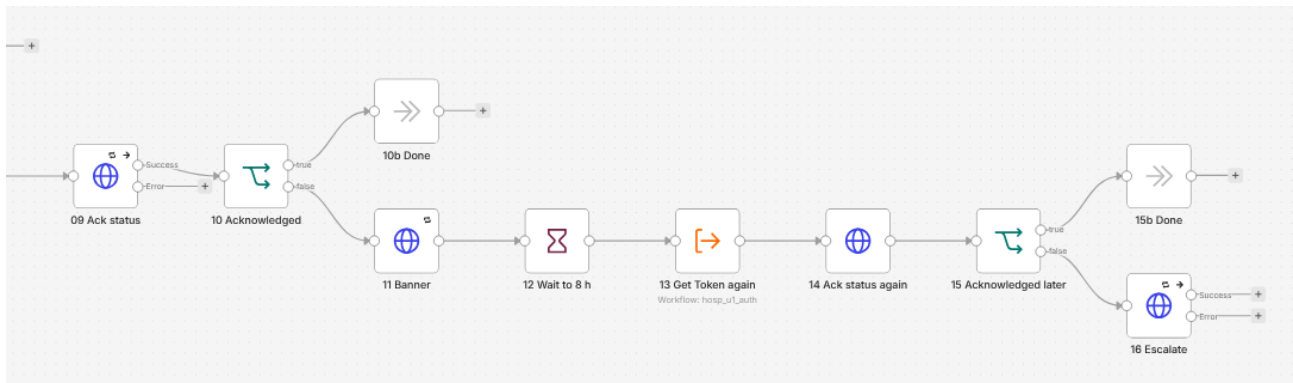


Figure 9: hosp-submission-watch, second half: banner, wait to 8 hours, escalate

hosp-query-intake

Trigger: webhook `query/intake`, when the insurer sends a question.

What it does:

- Get the query, and ask the **CrewAI query triage agent** what kind of query it is. Wait for the answer.
- Send the triage to the API.
- If it's simple and safe (fewer than 3 rounds, no escalation risk), start a reply draft automatically. Otherwise, give it to a person.

Why we need it: Queries delay discharge. Sorting them at once, and drafting a reply, saves hours.

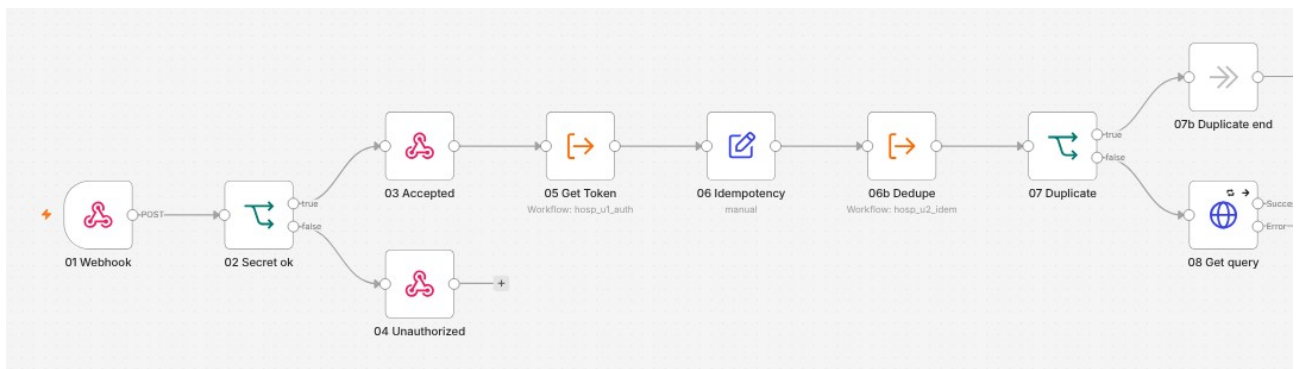


Figure 10: hosp-query-intake, first half

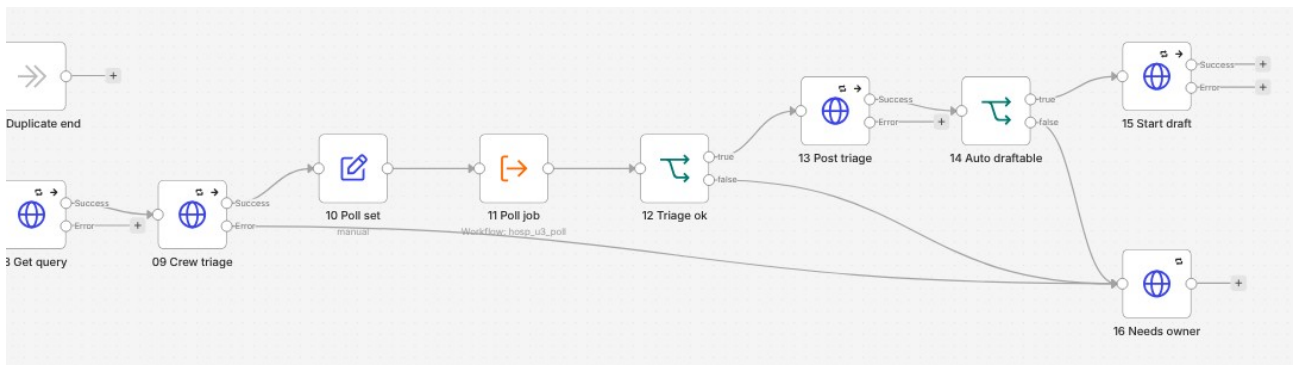


Figure 11: hosp-query-intake, second half: crew triage, auto-draft or owner

hosp-query-draft

Trigger: webhook `query/draft`.

What it does:

- Ask the **CrewAI reply writer** to draft the answer, using only facts from the case.
- If the agents are not available, record that, and a person writes the reply.

Why we need it: The officer starts from a good draft instead of a blank page. They always review it before it is sent.

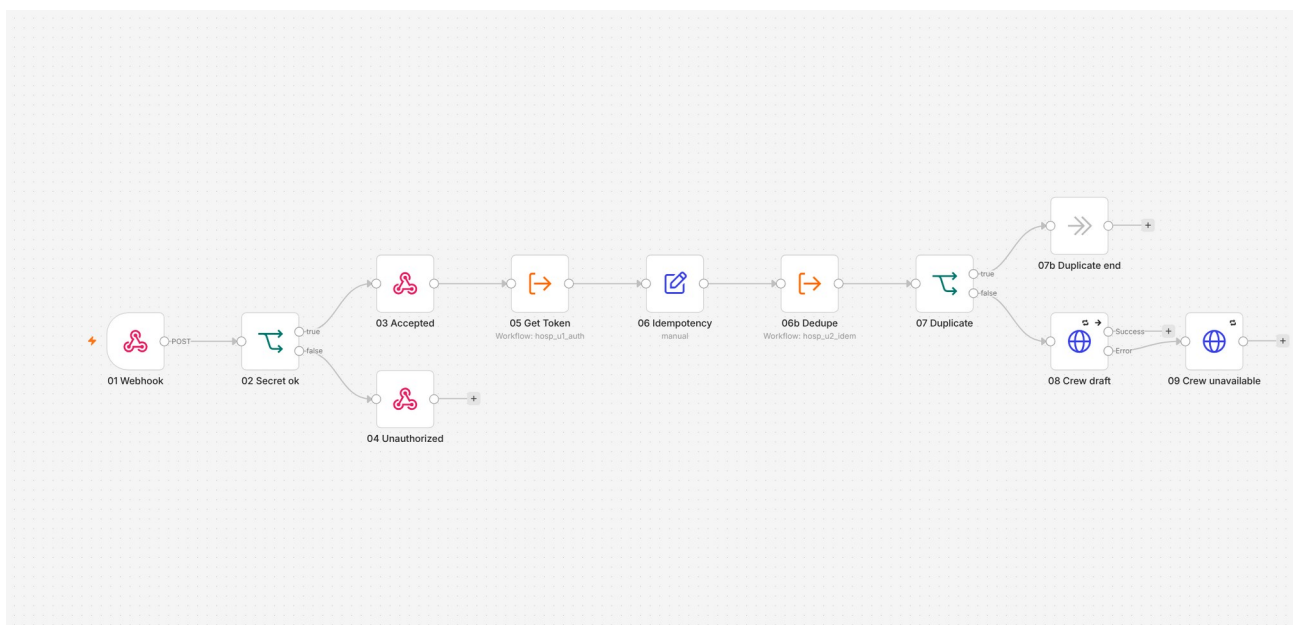


Figure 12: hosp-query-draft

hosp-query-sla-check

Trigger: a clock every hour (and webhook `jobs/query-sla`).

What it does:

- Ask the API to mark every query whose reply time has run out as **overdue**.

Why we need it: Insurers give a deadline for replies. Missing it can delay or reject the claim.

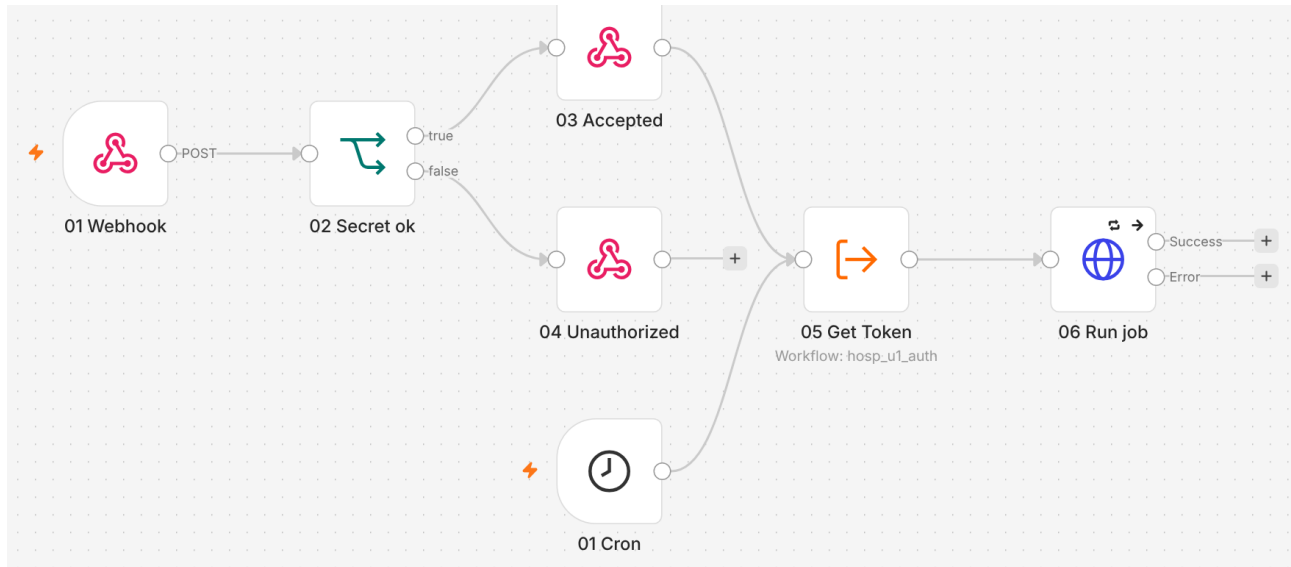


Figure 13: `hosp-query-sla-check`

hosp-reminders

Trigger: a clock every 5 minutes (and webhook `jobs/reminders`).

What it does:

- Get the reminders that are due, and send them to the screen 10 at a time.
- Email is skipped in this setup.

Why we need it: Staff are busy. Small reminders keep claims moving.

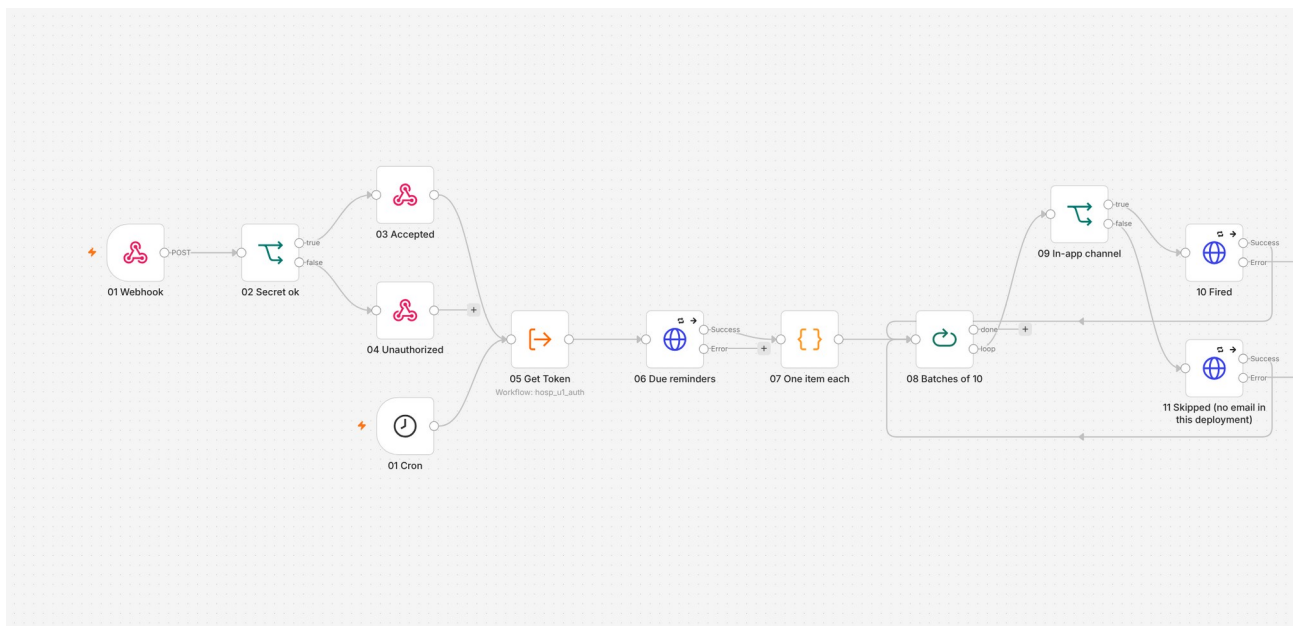


Figure 14: hosp-reminders

hosp-stuck-sweeper

Trigger: a clock every 2 minutes (and webhook `jobs/sweeper`).

What it does:

- Restart documents that got stuck while being read.
- If messages to the insurer are stuck in the outbox, tell the admin.

Why we need it: Things sometimes stop halfway. The sweeper cleans up so nothing is forgotten.

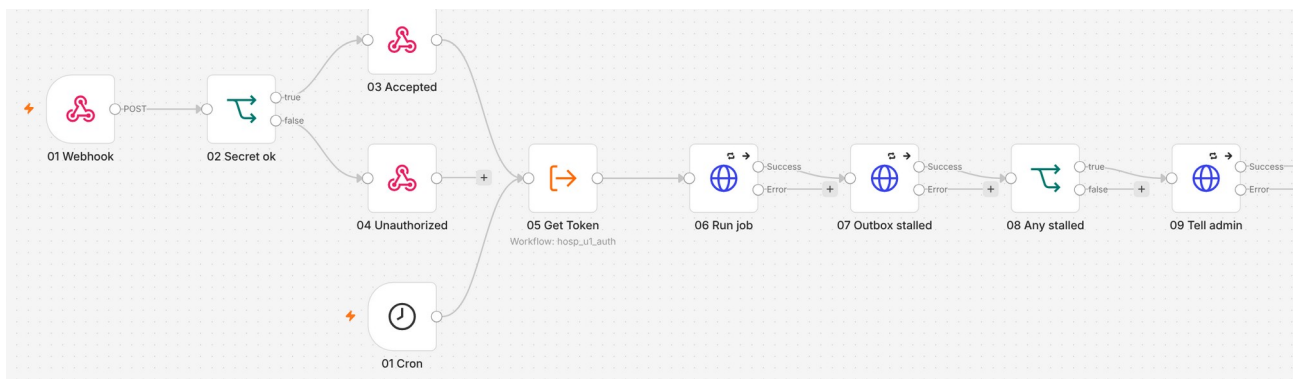


Figure 15: hosp-stuck-sweeper

hosp-global-error

Trigger: error trigger: any hospital workflow crashes.

What it does:

- Get a token and report the crash to the API's operations log.

Why we need it: One place to see every failure, instead of hunting through each workflow.

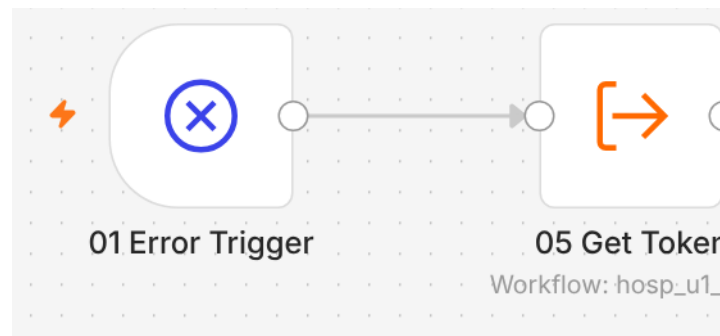


Figure 16: hosp-global-error

Helpers: util-auth-token, util-idempotency, util-poll-job

Trigger: called by other workflows (sub-workflow).

- **util-auth-token** gets a login token from Keycloak.
- **util-idempotency** asks the API 'have we seen this message?' (the API uses Redis for this).
- **util-poll-job** keeps asking 'is the job done yet?' but only up to a time limit (`max_wait_s`), so it never waits forever.

Why we need them: the same small jobs are needed in many workflows. Writing them once means fewer mistakes.

5. Insurer workflows (19)

The insurer side has its own n8n, on port 5689. It runs verification, decisions, queries, escalations and payment. Each workflow has a yellow sticky note that says what it is for.

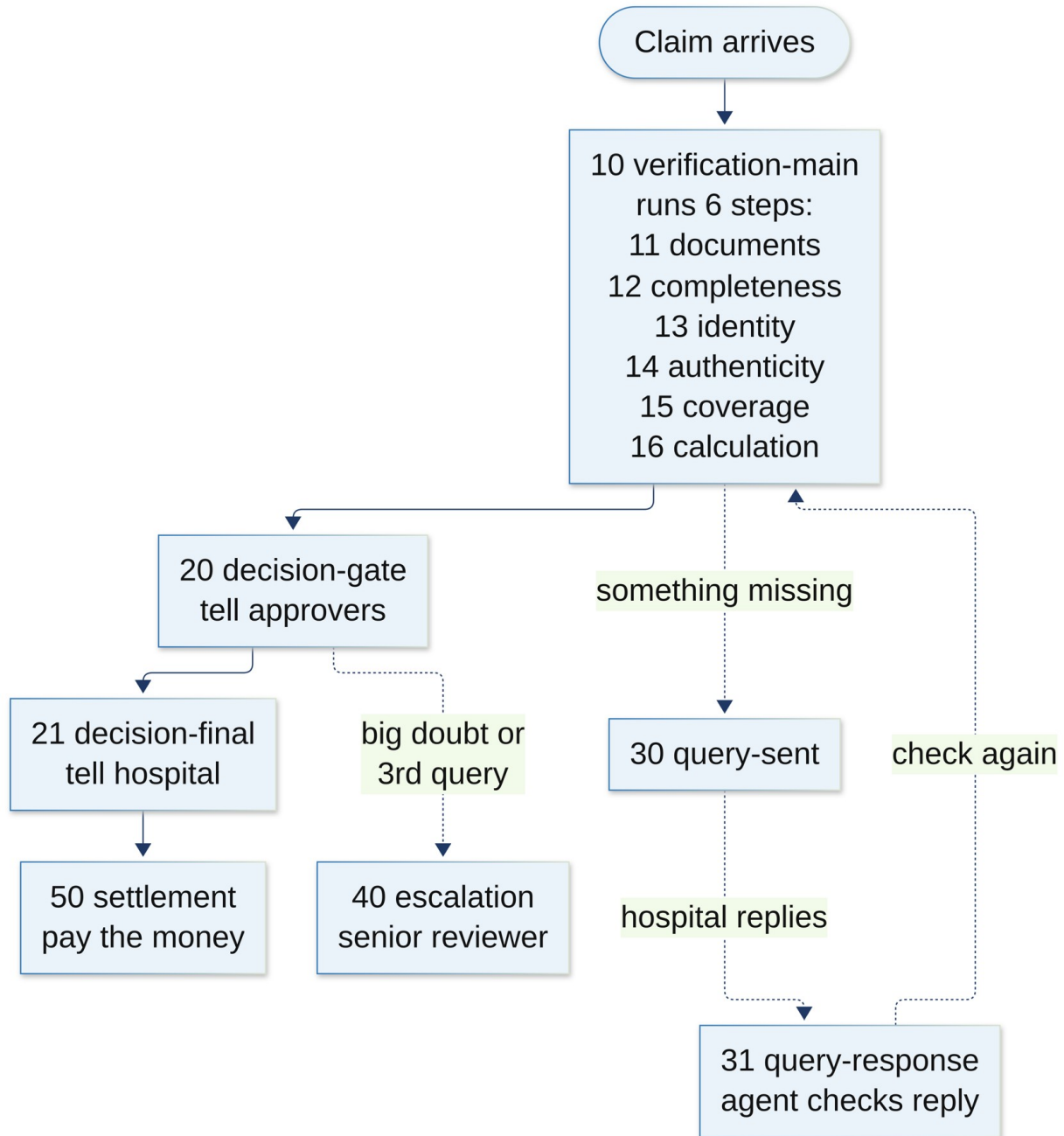


Figure 17: Insurer workflows: how a claim moves through them

5.1 The helper workflows (00-04)

Workflow	Trigger	What it does, and why
00 common error handler	Error trigger	When anything crashes: clean out private data, raise an alert, mark the run failed. One place to catch every failure

Workflow	Trigger	What it does, and why
01 get token	Sub-workflow	Get a Keycloak login token and keep it for 4 minutes
02 call API	Sub-workflow	Call the insurer API with a token and a unique key. Retry after 2, 8 and 30 seconds. 'Already running' counts as success
03 call crew	Sub-workflow	Call the CrewAI agents. Wait up to 100 seconds. If busy, wait and try again (up to 3 times). Bad answer → 'invalid output', down → 'unavailable'
04 emit event	Sub-workflow	Send an event to the API, which pushes it live to the reviewer's screen

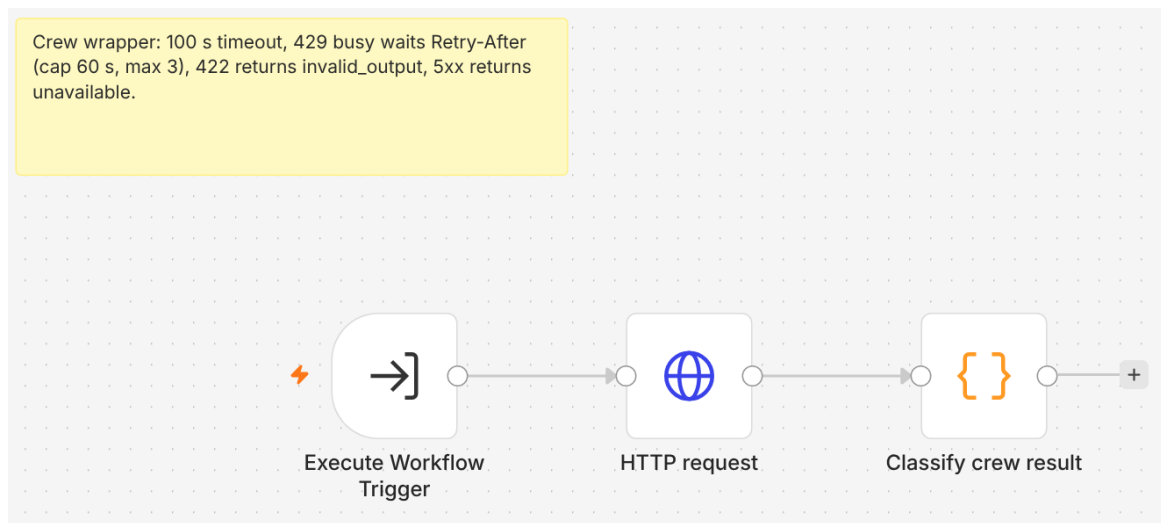


Figure 18: 03_call_crew: the bridge from n8n to the CrewAI agents

10 verification-main

Trigger: webhook `verification-start`, when a claim arrives from the hospital.

What it does:

- Check the signature, reply 202.
- Create a 'verification run'. If this claim already has a run, stop (duplicate).
- Run the six steps in order: **11 documents**, **12 completeness**, **13 identity**, **14 authenticity**, **15 coverage**, **16 calculation**.
- Finish the run. If it's ready for a decision, announce **decision.ready**. If something is missing, the API sends a query instead.

Why we need it: This is the main checklist for every claim. It makes sure every check is done, in the right order, every time.

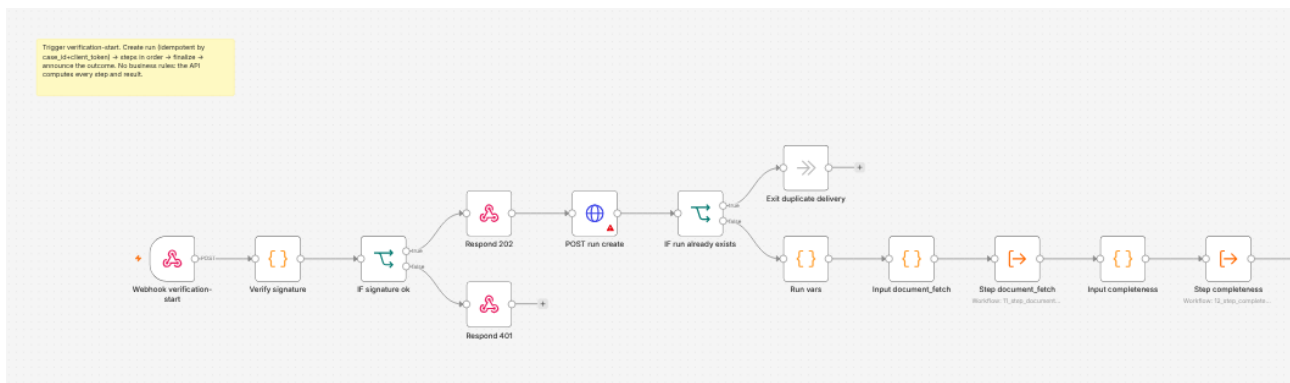


Figure 19: 10_verification_main, first half: signature, create run, first steps

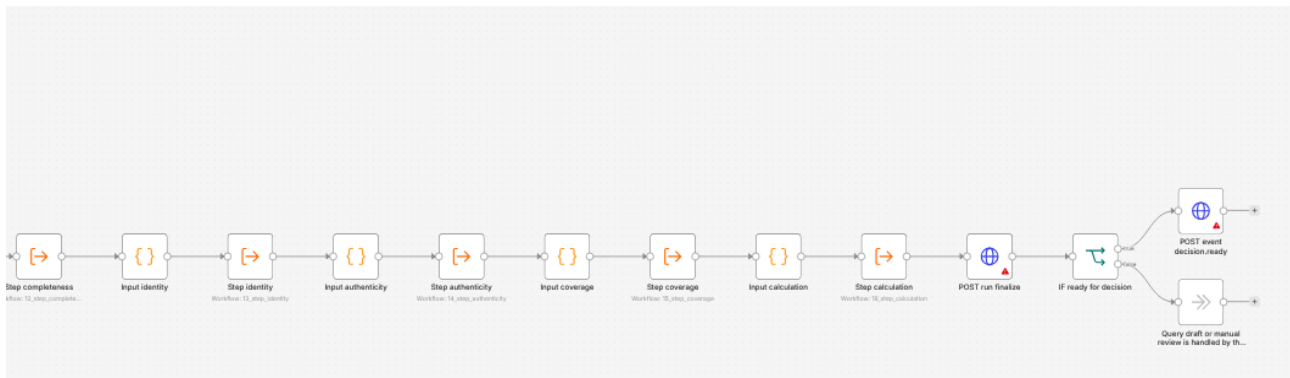


Figure 20: 10_verification_main, second half: remaining steps, finalise, decision.ready

5.2 The six verification steps (11-16)

Trigger: each one is called by 10 verification-main. They all follow the same shape: **start the step → get the facts → (ask the agent, if this step uses one) → send the result to the API**. The API does the fixed rules; the agent adds the reading and explaining.

Step	What it checks	Agent?
11 document fetch	Are all the claim's documents downloaded and read? Checks every 3 seconds until done	No
12 completeness	Are all required documents there for this kind of treatment?	No
13 identity	Is this the right patient? Name, date of birth, policy number vs the member record	Yes: identity agent
14 authenticity	Are the bills genuine? Stamp, totals, duplicates, signs of editing	Yes: authenticity agent
15 coverage	What does the policy cover for this treatment? Quotes the clauses	Yes: coverage agent (with RAG)
16 calculation	Map bill lines, then the calculation engine works out the payable amount	Yes: calculation mapper

If an agent is down or gives a bad answer, the step still finishes with a 'failure' note, the run continues, and the claim will go to a person. This rule is important: one broken agent never stops the whole claim.

Step identity: start → (context → crew, if agent-backed) → post the result; the API computes the deterministic part. Failure policy 6.15: crew unavailable or invalid output → post a failure body, the run continues.

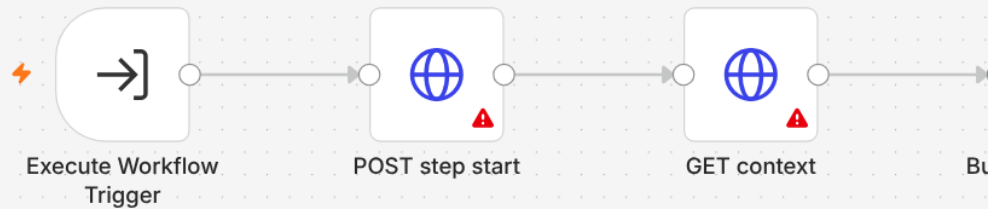


Figure 21: 13_step_identity: start, get context, build the crew request

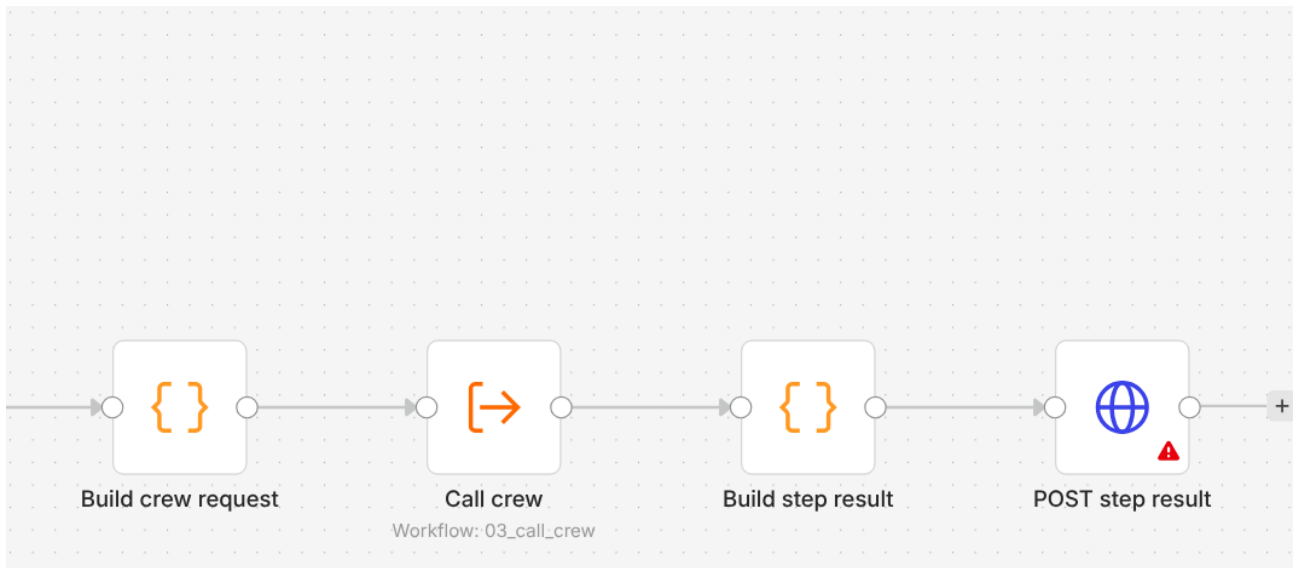


Figure 22: 13_step_identity: call the crew, build and post the result

Step document_fetch: start → (context → crew, if agent-backed) → post the result; the API computes the deterministic part. Failure policy 6.15: crew unavailable or invalid output → post a failure body, the run continues.

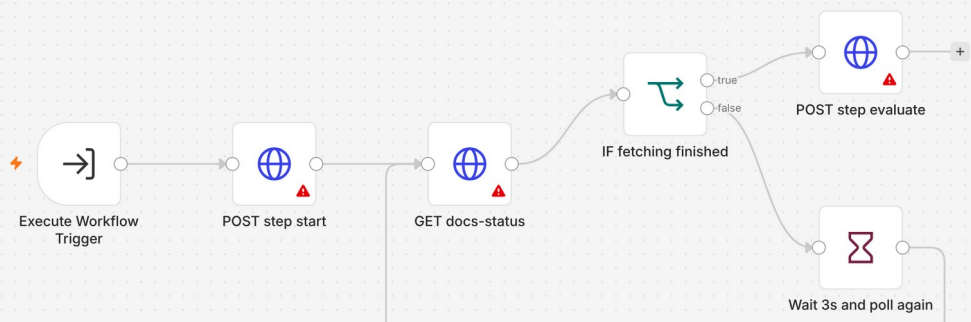


Figure 23: 11_step_document_fetch: a loop that checks every 3 seconds

20 decision-gate

Trigger: webhook `decision-task-open`, when a claim needs a human decision.

What it does:

- Send an **approval.requested** event, which pops up on the approvers' screens.
- Reminders for slow approvals come from the clock workflow (60), not from a long wait here.

Why we need it: Approvers must know at once that a claim is waiting for them.

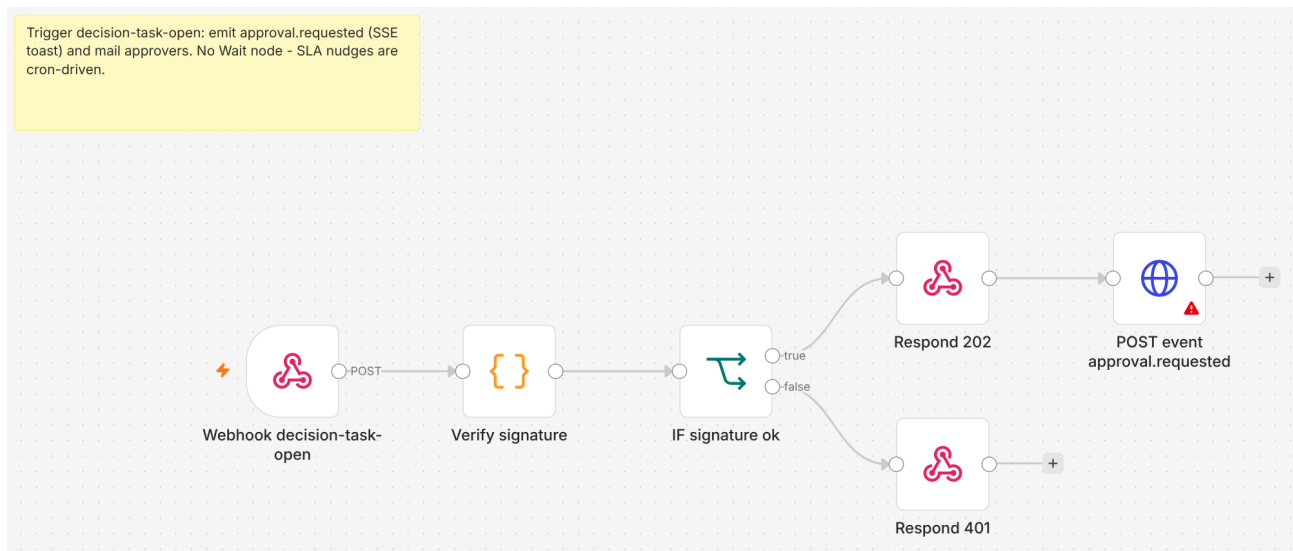


Figure 24: 20_decision_gate

21 decision-final

Trigger: webhook `decision-final`, when the decision is made (by the system or a person).

What it does:

- Ask the API to send the decision back to the hospital (signed).
- If the claim is approved or partly approved, start the payment (50).

Why we need it: The hospital learns the result straight away, and payment starts without anyone pressing a button.

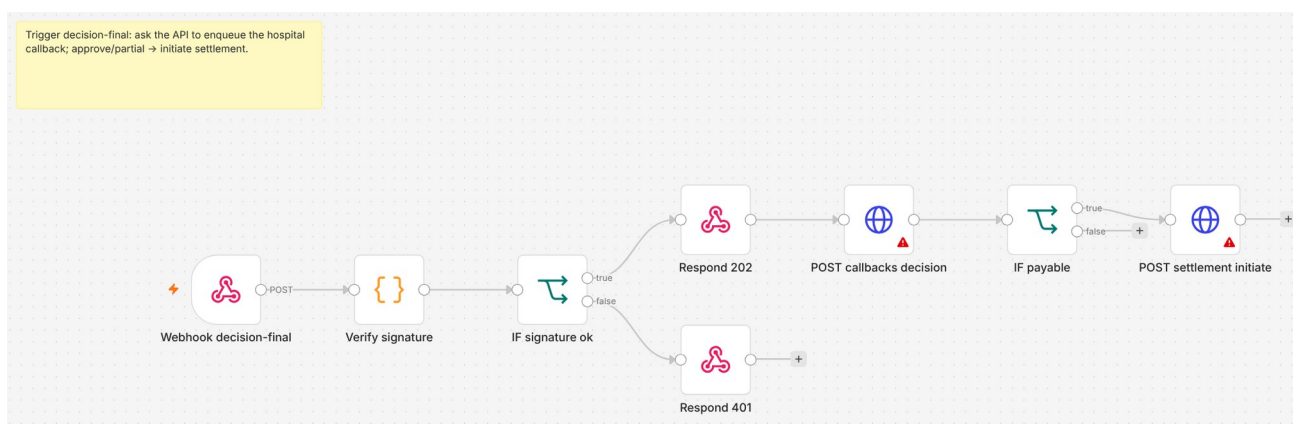


Figure 25: 21_decision_final

30 query-sent

Trigger: webhook **query-sent**, when the insurer sends a question to the hospital.

What it does:

- Send a **query.sent** event to the screens. Reminders and time-outs are handled by the clock workflow (60).

Why we need it: Everyone can see that the claim is now waiting for the hospital.

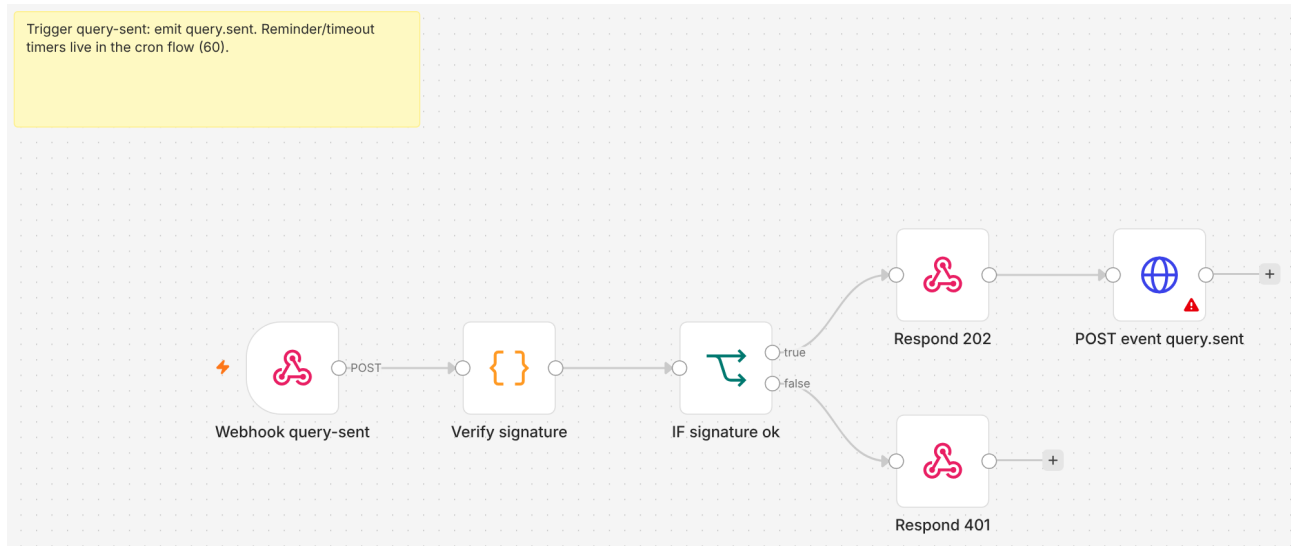


Figure 26: 30_query_sent

31 query-response

Trigger: webhook **query-response**, when the hospital replies.

What it does:

- Get the facts, ask the **CrewAI triage agent**: does this reply answer the query?
- Send the result to the API. The fixed rules decide.
- If the answer is good, start verification again (by calling its own webhook 10).

Why we need it: Checking replies quickly means the claim moves on the same day.

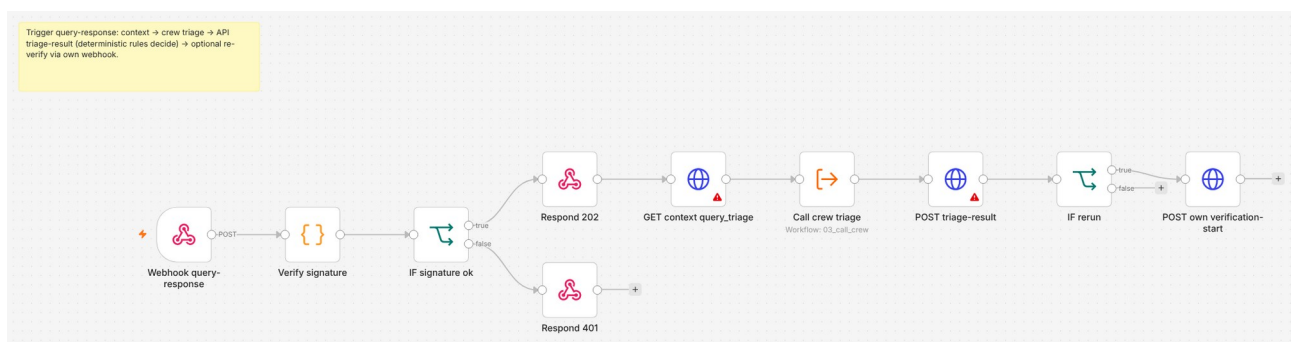


Figure 27: 31_query_response

40 escalation

Trigger: webhook **escalation-raised**, for example after 3 query rounds or a serious doubt.

What it does:

- Get the full escalation pack and send **escalation.opened** to the senior reviewers.

Why we need it: Some claims need a senior person. This makes sure they get the full story at once.

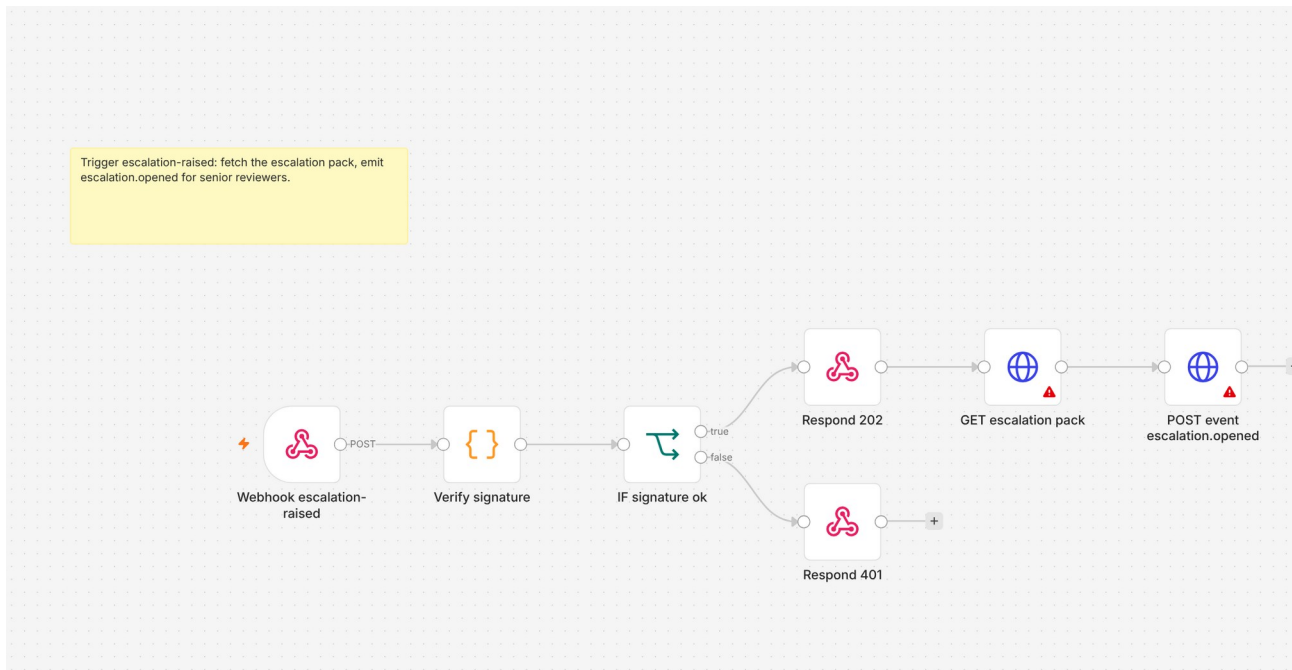


Figure 28: 40_escalation

50 settlement

Trigger: webhook **settlement-initiate**, from 21 decision-final.

What it does:

- Ask the API to pay through the bank simulator.
- Every 30 seconds, check if the payment is settled or failed (up to 12 times), in case the bank's own callback is lost.

Why we need it: The money must actually arrive. Checking makes sure a lost message doesn't leave the payment unknown.

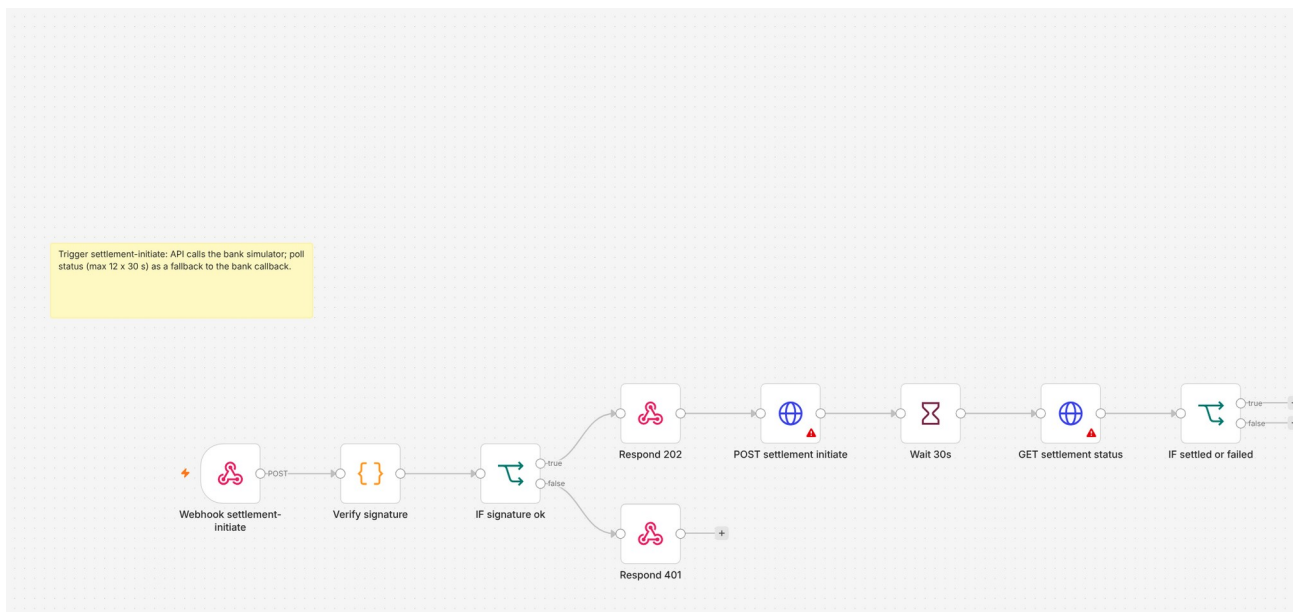


Figure 29: 50_settlement

60 cron SLA and housekeeping

Trigger: two clocks: every 5 minutes, and every night at 2 am.

What it does:

- **Every 5 minutes:** find queries that are due for a reminder or have timed out, and push any messages stuck in the outbox.
- **Every night at 2 am:** check the whole audit chain is unbroken, refresh the settings cache, and clean out old duplicate-check keys.

Why we need it: Time-based jobs in one place. The nightly audit check proves nobody has secretly changed an old record.

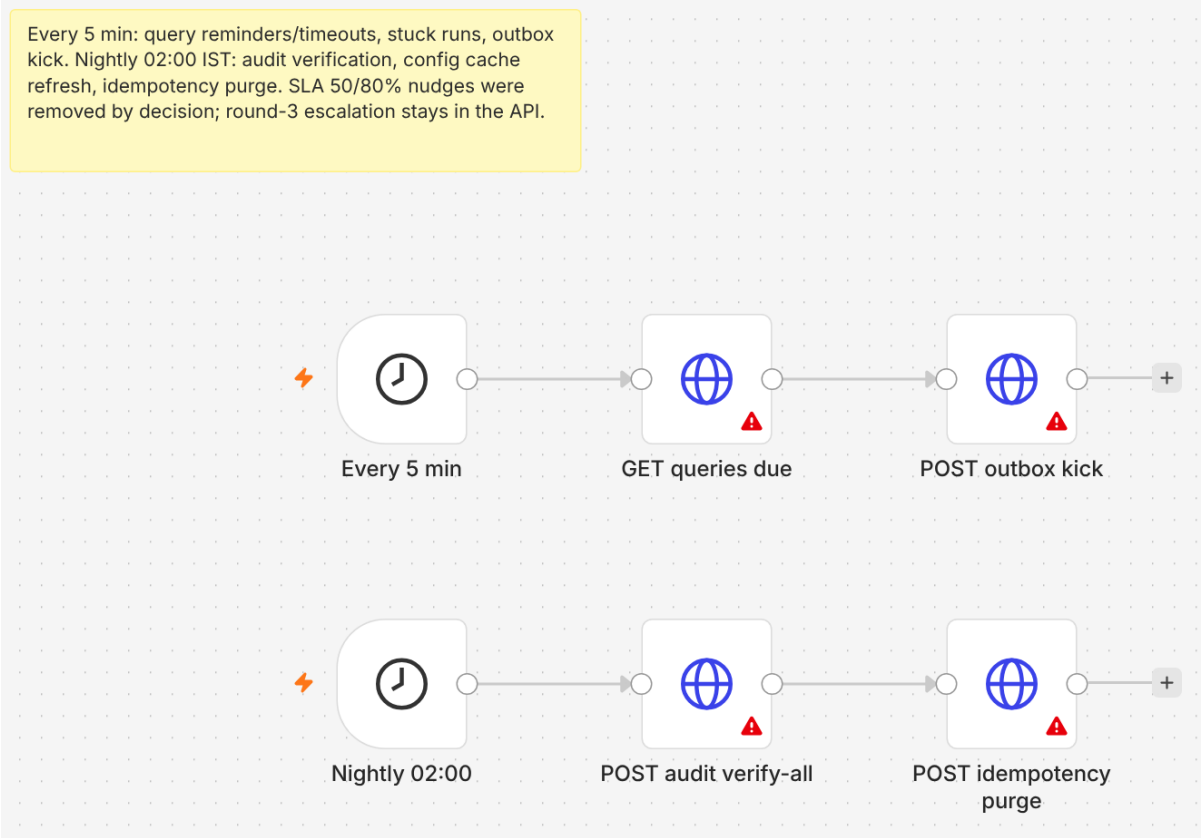


Figure 30: 60_cron_sla_and_housekeeping

6. How the workflows are made and loaded

We don't draw the workflows by hand in the editor. We write them in Python, so they are saved in Git, reviewed like code, and can be rebuilt any time.

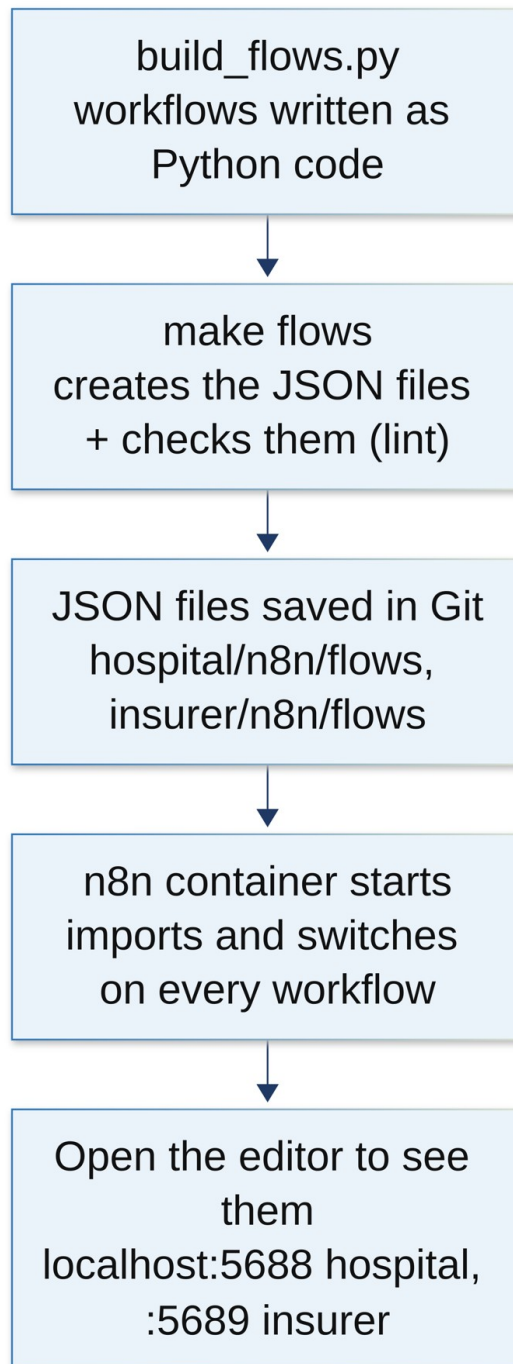


Figure 31: From code to running workflows

```

make flows them          # build hospital workflows from code and check
make up-n8n              # start hospital n8n (port 5688), workflows
switch on
make up-insurer-n8n     # start insurer n8n (port 5689)
make n8n-test            # test the workflows in a real n8n container

```

A test fails if someone edits the JSON by hand and it no longer matches the code. In `make demo`, the hospital workflows always run. The insurer ones run when `DEMO_ORCH=n8n` is set; otherwise the insurer API runs the same steps itself.

7. Workflow settings (parameters)

Setting	Value	What it means
N8N_WEBHOOK_SECRET	generated	The shared password the hospital API must send to start a workflow
HMAC signature	generated keys	The insurer's webhooks only accept signed messages
X-Idempotency-Key	one per message	A unique ID so the same message is never handled twice
Token cache	4 minutes	How long a login token is reused
API retries	2 s, 8 s, 30 s	Waits between tries when an API call fails
Crew timeout	100 s	How long n8n waits for the agents
Crew busy (429)	wait up to 60 s, max 3 times	If the agents are busy, wait as they ask, then try again
Document poll	every 3 s	How often step 11 checks if documents are ready
Settlement poll	every 30 s, 12 times	How often payment status is checked
Submission watch	15 min, then 8 h	When to show a banner, and when to raise an alert
Clocks	2 min, 5 min, 1 h, 2 am	Sweeper; reminders, stuck builds, query timers; late replies; nightly audit
Reminder batch	10	How many reminders are sent at a time
Ports	5688, 5689	Hospital and insurer n8n editors

8. Questions the examiner may ask about the workflows

Q: Why not do all of this in Python code?

We could, but then the waiting, retrying and reminders would be hidden in code. n8n shows the process as a picture, handles waiting for hours or days, and has retries and error handling built in.

Q: What if n8n is down?

The API keeps the real state. When n8n comes back, the clock workflows (sweeper, stuck builds) pick up anything left behind. Nothing is lost, because n8n never holds the only copy of anything.

Q: What if the same message arrives twice?

The duplicate check (idempotency key) stops the second one. And even if it got through, the API refuses a step that doesn't fit the claim's current state.

Q: Why does the webhook reply before the work is done?

So the caller isn't kept waiting. Reading a document or asking an agent can take a minute. The API just needs to know the job was accepted.

Q: How do workflows talk to the agents?

Through an HTTP call to the crew service: `03_call_crew` on the insurer side, and the crew calls inside query intake and draft on the hospital side. The agents answer, and n8n hands the answer to the API.

Q: Why are there two n8n's?

Because the hospital and the insurer are two separate companies. Each one runs its own workflows, on its own network, with its own secrets.

Q: Why are the workflows written in Python and not drawn?

So they are saved in Git, can be reviewed and tested, and can be rebuilt exactly. The editor is still there to look at them.